01 - Veri Özeti

Amaç ve Kapsam

Bu notebook, tezin makine öğrenmesi ve yapay zeka temeli üzerine inşa edilen mühendislik (feature engineering) çerçevesini tanıtmayı ve ve ön kritik keşfetsel veri analizi (Exploratory Data Analysis - EDA) sürecini. Alibaba PAI 100K GPU iş İzralelerinin ham kullanıcıyasının incelenmesi, modellerden (Random Forest, LightGBM, CNN, LSTM vb.) hangi sinyalin eğilimlendiğine ve bu verilerin istatistiksel geçerliliğine (validity) dair nesnel kanıtların tesisini oluşturur.

Bu aşamanın gerçekleştirdiği ana hedefler, tezin ilerleyen aşamalarında mimari kararlar doğurabilecek iş gelişkisi hem yaşamaktadır:

- **Veri Bütünlüğü ve Veri Kayıp (Missing Data):** Kimlikleri eksik ham iş yüklemleri eksik veya gaspeyn dağılımı eşlenek, derin öğrenme modellerinin (örneğin LSTM) anormal veya boşluk gradyanına (exploding gradients) maruz kalmasını engelleme (veya sahte veri üretmeyi) amacıyla ön temizlik ve veri tamamlama (imputation) stratejilerinin belirlenmesi.
- **Kategorik Değişkenlerin Kardinalitesi:** Sunucu kümesi (cluster) gibi kategorik değişkenin analiz edilerek One-Hot kodlama yerine neden Native Categorical (LightGBM) yaklaşımının denenmesi gerektiğinin istatistiksel olarak kanıtlanması.
- **Aykırı Bilgi (Outlier) Analizi:** Sunucuda İşlemcinin eğitimlerinde yaşanan olukların (IOPS) tamamen, hoşu İnsanların ve/veyanın analizini yapacak modeller için hangi gizli (predictive power) taşıdığın belirlenmesi.
- **Hem Zaman Serisi Kalıbının Testi:** Zaman damgası verilerin anlıştığı yapıyan doğrulayacak, istatistiksel eşamanlamanın (`mann_whitney_test_analysis.csv`) mahsuna gösterek ciler olay kurulduğunun (event based) kronolojik güvenilirliğini sağlamak.

Kullanılan kütüphane çekirdeği, hem sunucu tarafına ulaşı iş Yıldız Kandelanoçunu (Notebook 02) ve Cluster kümelendiği (Notebook 03) aşamalarına veri ulaşabilmesi (data drivers) sağlıyan bir köprü kurmaktadır.

```
# Core libraries
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Paths
from src.data_loading import load_main_sample
from src.feature_engineering import build_log_table_from_sample

# Plot defaults
sns.set_theme(style="whitegrid", palette="muted")
pd.pd.set_option("display.max_columns", 50)
plt.rcParams["figure.figure.dpi"] = 100
plt.rcParams["figure.figsize"] = (10, 4)
plt.rcParams["axes.titlesize"] = 14
plt.rcParams["axes.labelsize"] = 12
plt.rcParams["font.family"] = "DejaVu Sans"

print("Çalıştırıyor: kici ipynb\n")
[Warning] kici ipynb\n
```

Kurulum doğrulandı. Proje kök dizini Python yolunda eklendiş, `load_log_table_from_sample` çalıştırılan veri dizinleri doğrulanmıştır. Notebook tekrarlanabilir biçimde çalıştırıyor hazırdır.

1. Veri Yüklenmesi

Ham veri, Alibaba PAI 2020 GPU Cluster Trace'inin yalın bir `.csv` dosyasıdır. Bu dosyanın çalışma çıkışına sürec bütünlüğünün ölçülebilmesi ve ilerleyen adımlarda yapılan dönüşümlerin izlenebilmesi amacıyla, ham veri ve ilk temel özellikler aşağıdaki hücrede yüklenmiş ve boyutlarına ilişkin kısa bir özet çıkarılmıştır.

```
# 1. Load the sample
df = load_main_sample()
df.shape, df.head(2)
print(f"Shape: {df.shape}")
df.info()
[Shape: (1,000,000, 9)
<class pandas.core.frame.DataFrame>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 9 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   job_id     1000000 non-null  int64
 1   inst_id    1000000 non-null  int64
 2   submit_time 1000000 non-null  int64
 3   inst_start 1000000 non-null  int64
 4   inst_end   1000000 non-null  int64
 5   gpu_type   1000000 non-null  object
 6   gpu_spec   1000000 non-null  object
 7   duration   1000000 non-null  int64
 8   wait_time  1000000 non-null  int64
dtypes: int64(7), object(2)
memory usage: 6.7 GB
```

Sütun İsimleri ve tod yapıdan raporlandı. Sayısal İstatistikler (`.describe()`) doğru tipte olduğu doğrulanmıştır. `inst_time` sütununda eksik değer olan bir sorunlu hücrede dolaylandırılmıştır.

```
# 2. Missing value scanner
missing = df.isna().sum()
missing[missing > 0] if missing.sum() > 0 else "No missing values."
missing_percent = pd.DataFrame({"missing_count": missing, "missing_%": missing.pct*100})
missing_percent[missing_percent.missing_count > 0].sort_values("missing_%", ascending=False)
[No missing values.
# 3. Data quality checks
# Negative durations
neg = (df.inst_end - df.inst_start) < 0
print(f"Negative duration: {neg.sum:,} ({neg.mean()*100:.4f%})")
[Negative duration: 2,018(0.0202%)
# 4. Duration statistics
dur = df.duration
dur.describe(percentiles=[0.01, 0.05, 0.50, 0.95, 0.99])
[count    1000000.0
mean      3600.0
std       0.0
min       3600.0
1%        3600.0
5%        3600.0
50%       3600.0
95%       3600.0
99%       3600.0
max       3600.0
dtype: float64
# 5. Wait time
wait = df.wait_time
wait.describe(percentiles=[0.01, 0.05, 0.50, 0.95, 0.99])
[count    1000000.0
mean      0.0
std       0.0
min       0.0
1%        0.0
5%        0.0
50%       0.0
95%       0.0
99%       0.0
max       0.0
dtype: float64
# 6. Quick sanity summary
sanity = {
    "rows": df.shape[0],
    "columns": df.shape[1],
    "unique_jobs": df.job_id.nunique(),
    "unique_instances": df.inst_id.nunique(),
    "missing_total": int(missing.sum),
    "negative_durations": int(neg.sum),
    "duration_mean": dur.mean(),
    "duration_std": dur.std(),
    "wait_mean": wait.mean(),
    "wait_max": wait.max(),
}
pd.Series(sanity)
[rows                 1000000
columns                    9
unique_jobs           ...
unique_instances      ...
missing_total             0
negative_durations     2018
duration_mean          3600
duration_std           ...
wait_mean              ...
wait_max               ...
dtype: object
```

Sütun İsimleri ve tod yapıdan raporlandı. Sayısal İstatistikler (`.describe()`) doğru tipte olduğu doğrulanmıştır. `inst_time` sütununda eksik değer olan bir sorunlu hücrede dolaylandırılmıştır.

2. Missing Value Scanner

Ham veri kümesinin bütünlüğünün ilk ölçümü olan bu hücre, hangi sütunlarda ne kadar eksik veri bulunduğunu sayısal olarak ortaya çıkarmaktadır. Bulgular daha sonraki hücrelerde ayrıntılandırılacak olsa da, burada elde edilen sonuç, veri temizliği ve tamamlama (imputation) stratejilerinin hangi sütunlara odaklanacağına doğrudan girdi sağlamaktadır.

```
# 2. Missing value scanner
missing = df.isna().sum()
missing[missing > 0] if missing.sum() > 0 else "No missing values."
missing_percent = pd.DataFrame({"missing_count": missing, "missing_%": missing.pct*100})
missing_percent[missing_percent.missing_count > 0].sort_values("missing_%", ascending=False)
[No missing values.
# 3. Data quality checks
# Negative durations
neg = (df.inst_end - df.inst_start) < 0
print(f"Negative duration: {neg.sum:,} ({neg.mean()*100:.4f%})")
[Negative duration: 2,018(0.0202%)
# 4. Duration statistics
dur = df.duration
dur.describe(percentiles=[0.01, 0.05, 0.50, 0.95, 0.99])
[count    1000000.0
mean      3600.0
std       0.0
min       3600.0
1%        3600.0
5%        3600.0
50%       3600.0
95%       3600.0
99%       3600.0
max       3600.0
dtype: float64
# 5. Wait time
wait = df.wait_time
wait.describe(percentiles=[0.01, 0.05, 0.50, 0.95, 0.99])
[count    1000000.0
mean      0.0
std       0.0
min       0.0
1%        0.0
5%        0.0
50%       0.0
95%       0.0
99%       0.0
max       0.0
dtype: float64
# 6. Quick sanity summary
sanity = {
    "rows": df.shape[0],
    "columns": df.shape[1],
    "unique_jobs": df.job_id.nunique(),
    "unique_instances": df.inst_id.nunique(),
    "missing_total": int(missing.sum),
    "negative_durations": int(neg.sum),
    "duration_mean": dur.mean(),
    "duration_std": dur.std(),
    "wait_mean": wait.mean(),
    "wait_max": wait.max(),
}
pd.Series(sanity)
[rows                 1000000
columns                    9
unique_jobs           ...
unique_instances      ...
missing_total             0
negative_durations     2018
duration_mean          3600
duration_std           ...
wait_mean              ...
wait_max               ...
dtype: object
```

Sütun İsimleri ve tod yapıdan raporlandı. Sayısal İstatistikler (`.describe()`) doğru tipte olduğu doğrulanmıştır. `inst_time` sütununda eksik değer olan bir sorunlu hücrede dolaylandırılmıştır.

3. Veri Kalitesi Kontrolleri

Bu hücre, veri kümesinin temel kalite ölçütlerini sistematik biçimde ölçer. Negatif çalışma süreleri, sıfır bekleme süreleri ve eksik veri oranlarının ortaya konmasıyla, ham veri kümesinin hangi yönlerinin ön temizlik ve dönüştürme adımlarına ihtiyaç duyacağına dair nesnel kanıtlar sağlanmıştır. Bu bulgular, ilerleyen bölümlerde kullanılacak olan veri temizleme ve dönüştürme stratejilerinin temel girdisini oluşturur.

```
# 4. Duration statistics
dur = df.duration
dur.describe(percentiles=[0.01, 0.05, 0.50, 0.95, 0.99])
[count    1000000.0
mean      3600.0
std       0.0
min       3600.0
1%        3600.0
5%        3600.0
50%       3600.0
95%       3600.0
99%       3600.0
max       3600.0
dtype: float64
# 5. Wait time
wait = df.wait_time
wait.describe(percentiles=[0.01, 0.05, 0.50, 0.95, 0.99])
[count    1000000.0
mean      0.0
std       0.0
min       0.0
1%        0.0
5%        0.0
50%       0.0
95%       0.0
99%       0.0
max       0.0
dtype: float64
# 6. Quick sanity summary
sanity = {
    "rows": df.shape[0],
    "columns": df.shape[1],
    "unique_jobs": df.job_id.nunique(),
    "unique_instances": df.inst_id.nunique(),
    "missing_total": int(missing.sum),
    "negative_durations": int(neg.sum),
    "duration_mean": dur.mean(),
    "duration_std": dur.std(),
    "wait_mean": wait.mean(),
    "wait_max": wait.max(),
}
pd.Series(sanity)
[rows                 1000000
columns                    9
unique_jobs           ...
unique_instances      ...
missing_total             0
negative_durations     2018
duration_mean          3600
duration_std           ...
wait_mean              ...
wait_max               ...
dtype: object
```

Sütun İsimleri ve tod yapıdan raporlandı. Sayısal İstatistikler (`.describe()`) doğru tipte olduğu doğrulanmıştır. `inst_time` sütununda eksik değer olan bir sorunlu hücrede dolaylandırılmıştır.

4. Duration Statistics

Bu hücre, veri kümesinin temel kalite ölçütlerini sistematik biçimde ölçer. Negatif çalışma süreleri, sıfır bekleme süreleri ve eksik veri oranlarının ortaya konmasıyla, ham veri kümesinin hangi yönlerinin ön temizlik ve dönüştürme adımlarına ihtiyaç duyacağına dair nesnel kanıtlar sağlanmıştır. Bu bulgular, ilerleyen bölümlerde kullanılacak olan veri temizleme ve dönüştürme stratejilerinin temel girdisini oluşturur.

```
# 5. Wait time
wait = df.wait_time
wait.describe(percentiles=[0.01, 0.05, 0.50, 0.95, 0.99])
[count    1000000.0
mean      0.0
std       0.0
min       0.0
1%        0.0
5%        0.0
50%       0.0
95%       0.0
99%       0.0
max       0.0
dtype: float64
# 6. Quick sanity summary
sanity = {
    "rows": df.shape[0],
    "columns": df.shape[1],
    "unique_jobs": df.job_id.nunique(),
    "unique_instances": df.inst_id.nunique(),
    "missing_total": int(missing.sum),
    "negative_durations": int(neg.sum),
    "duration_mean": dur.mean(),
    "duration_std": dur.std(),
    "wait_mean": wait.mean(),
    "wait_max": wait.max(),
}
pd.Series(sanity)
[rows                 1000000
columns                    9
unique_jobs           ...
unique_instances      ...
missing_total             0
negative_durations     2018
duration_mean          3600
duration_std           ...
wait_mean              ...
wait_max               ...
dtype: object
```

Sütun İsimleri ve tod yapıdan raporlandı. Sayısal İstatistikler (`.describe()`) doğru tipte olduğu doğrulanmıştır. `inst_time` sütununda eksik değer olan bir sorunlu hücrede dolaylandırılmıştır.

5. Wait Time

Bu hücre, veri kümesinin temel kalite ölçütlerini sistematik biçimde ölçer. Negatif çalışma süreleri, sıfır bekleme süreleri ve eksik veri oranlarının ortaya konmasıyla, ham veri kümesinin hangi yönlerinin ön temizlik ve dönüştürme adımlarına ihtiyaç duyacağına dair nesnel kanıtlar sağlanmıştır. Bu bulgular, ilerleyen bölümlerde kullanılacak olan veri temizleme ve dönüştürme stratejilerinin temel girdisini oluşturur.

```
# 6. Quick sanity summary
sanity = {
    "rows": df.shape[0],
    "columns": df.shape[1],
    "unique_jobs": df.job_id.nunique(),
    "unique_instances": df.inst_id.nunique(),
    "missing_total": int(missing.sum),
    "negative_durations": int(neg.sum),
    "duration_mean": dur.mean(),
    "duration_std": dur.std(),
    "wait_mean": wait.mean(),
    "wait_max": wait.max(),
}
pd.Series(sanity)
[rows                 1000000
columns                    9
unique_jobs           ...
unique_instances      ...
missing_total             0
negative_durations     2018
duration_mean          3600
duration_std           ...
wait_mean              ...
wait_max               ...
dtype: object
```

Sütun İsimleri ve tod yapıdan raporlandı. Sayısal İstatistikler (`.describe()`) doğru tipte olduğu doğrulanmıştır. `inst_time` sütununda eksik değer olan bir sorunlu hücrede dolaylandırılmıştır.

6. Quick Sanity Summary

Bu hücre, veri kümesinin temel kalite ölçütlerini sistematik biçimde ölçer. Negatif çalışma süreleri, sıfır bekleme süreleri ve eksik veri oranlarının ortaya konmasıyla, ham veri kümesinin hangi yönlerinin ön temizlik ve dönüştürme adımlarına ihtiyaç duyacağına dair nesnel kanıtlar sağlanmıştır. Bu bulgular, ilerleyen bölümlerde kullanılacak olan veri temizleme ve dönüştürme stratejilerinin temel girdisini oluşturur.

Özet

Bu notebook'ta gerçekleştirilen keşfetsel veri analizi (EDA), tezin tüm sonraki mühendislik kararlarını nesnel verilerle temellendiren kritik bulgular ortaya çıkarmıştır:

- **Kusursuz Veriler:** Bu ilgili kaynak hücreler (veriler) temiz bir şekilde ve hiçbir eksik veri (missing data) veya negatif çalışma süresi (negative duration) ile karşılaşılmamıştır. Bu durum, MSE yerine MAE/MAPE metriklerinin, doğrusal olmayan ağlaç ve derin öğrenme modellerinin oturumu kırılmadan (exploding gradients) nasıl daha kararlı bir eğitim süreci sağlayacağı gibi, ileri aşamalarda alınacak bazı mühendislik kararlarının temelini oluşturmaktadır.
- **Zamansal Kaydırma:** Zamansal eşleme, iletişim iş gücüyüne dönüşmüş, temporal özellik mühendisliğinin gerekliliğini ve kronolojik hataların önlenmesini yüksek kılmıştır.
- **Kullanıcı kompetansmanı:** Küçük bir kullanıcı grubunun ağır iş yükü üretmesi, `user` özelliğinin One-Hot kodlamasının boyut patlamasına neden olduğunu açıklamaktadır.

Bu bulgular, sonraki notebook'larda her çizerek kanıtı keşfi; veri temizlenmesi gerektiğinde (data cleaning) gerekli olma ihtiyacını ortaya koymaktadır.